Trabajo Práctico – Programación II

⚔️ Solo Leveling y la Programación Orientada a Objetos



Propósito pedagógico

En este trabajo práctico vamos a usar el universo de **Solo Leveling** para modelar un sistema orientado a objetos.

La idea no es que todos se conviertan en Sung Jin-Woo de un día para otro. La idea es que aprendan a:

- identificar objetos
- asignar responsabilidades
- modelar relaciones
- trabajar con colecciones
- diseñar clases con **alta cohesión** y **bajo acoplamiento**

Estructura del trabajo

- **Parte 1 – El despertar del cazador**
  Objetos, atributos, métodos, inventario y primeras relaciones.
- **Parte 2 – Especialización de cazadores**
  Tipos de cazadores, comportamiento específico y composición.
- **Parte 3 – Sistema de incursiones**
  Equipos, mazmorras, enemigos y desafío integral con ArrayList.

🟢 PARTE 1 – EL DESPERTAR DEL CAZADOR

Situación

Los portales comenzaron a aparecer y algunas personas despertaron poderes. Todavía no son leyendas, pero al menos ya pueden cargar un arma, una poción y entrar a una mazmorra sin desmayarse inmediatamente.

En esta primera parte vamos a modelar un cazador con su inventario y su relación con una mazmorra.

Objetivo técnico

Trabajar:

- clases
- atributos
- métodos
- relaciones entre objetos
- uso básico de ArrayList

Clases sugeridas

Cazador

Atributos sugeridos:

- nombre
- rango (E, D, C, B, A, S)
- poder
- inventario

Métodos sugeridos:

- mostrarInformacion()
- entrarMazmorra(Mazmorra mazmorra)
- mostrarInventario()

Item

Atributos sugeridos:

- nombre
- tipo
- valor

Métodos sugeridos:

- mostrarItem()

Inventario

Atributos sugeridos:

- ArrayList<Item> items

Métodos sugeridos:

- agregarItem(Item item)
- mostrarItems()
- cantidadItems()

Mazmorra

Atributos sugeridos:

- nombre
- dificultad

Métodos sugeridos:

- mostrarMazmorra()

Relación entre objetos

- Un Cazador **tiene un** Inventario
- Un Inventario **tiene muchos** Item
- Un Cazador puede interactuar con una Mazmorra

Preguntas guía

- ¿Qué sabe hacer el cazador?
- ¿Qué sabe hacer el inventario?
- ¿Quién debería mostrar los ítems?
- ¿Quién conoce su dificultad, el cazador o la mazmorra?

Clase Main sugerida – Parte 1

```
package parte1;

public class MainParte1 {
   public static void main(String[] args) {

      // 1. Crear algunos items
      // 2. Crear un inventario y agregar esos items
      // 3. Crear un cazador y asignarle el inventario
      // 4. Crear una mazmorra
      // 5. Mostrar informacion del cazador
      // 6. Mostrar su inventario
      // 7. Hacer que el cazador entre a la mazmorra

   }
}
```

Indicaciones para resolver la Parte 1

1. Crear al menos:
    - 1 cazador
    - 1 inventario
    - 3 items
    - 1 mazmorra
2. Mostrar por consola:
    - datos del cazador
    - contenido del inventario
    - mensaje narrativo al entrar a la mazmorra
3. Cuidar que cada clase haga lo que le corresponde.

"Un cazador sin inventario entra a una mazmorra con mucho entusiasmo y muy pocas probabilidades."



🟡 PARTE 2 – ESPECIALIZACIÓN DE CAZADORES

Situación

No todos los cazadores pelean igual.

Algunos curan, otros atacan desde lejos, otros aparecen desde las sombras y otros simplemente hacen explotar todo con dignidad mágica.

Ahora el sistema debe distinguir distintos tipos de cazadores.

Objetivo técnico

Trabajar:

- herencia
- especialización
- comportamiento diferenciado
- cohesión
- diseño más expresivo

Nuevas clases sugeridas

A partir de Cazador, crear:

- Sanador
- Asesino
- Mago
- Tanque

Idea de comportamiento

Sanador

Puede tener:

- mana
- capacidadCuracion

Método sugerido:

- curar()

Asesino

Puede tener:

- velocidad
- critico

Método sugerido:

- atacarSigilosamente()

Mago

Puede tener:

- mana
- poderMagico

Método sugerido:

- lanzarHechizo()

Tanque

Puede tener:

- defensa
- resistencia

Método sugerido:

- proteger()

Importante

Todos siguen siendo cazadores, pero no todos hacen exactamente lo mismo.

Ahí aparece una gran pregunta:

¿Qué debería estar en la clase Cazador y qué debería quedar en cada subclase?

Concepto a trabajar

Alta cohesión Cada subclase debe concentrarse en lo que realmente la define.
Bajo acoplamiento No queremos que Mago tenga lógica del Sanador, ni que el Tanque ande curando porque sí.

Clase Main sugerida – Parte 2

```
package parte2;

public class MainParte2 {
   public static void main(String[] args) {

      // 1. Crear un sanador
      // 2. Crear un asesino
      // 3. Crear un mago
      // 4. Crear un tanque
```

```
        // 5. Mostrar la informacion de cada uno
        // 6. Ejecutar al menos una accion propia de cada clase
        // 7. Reflexionar: que comparten y en que se diferencian

    }
}
```

Indicaciones para resolver la Parte 2

1. Crear al menos 4 cazadores de distinto tipo.
2. Cada uno debe tener:
    - atributos heredados de Cazador
    - atributos propios
    - al menos un método específico
3. Mostrar por consola acciones diferentes.

Preguntas guía

- ¿Todos los cazadores tienen nombre, rango y poder?
- ¿Todos los cazadores curan?
- ¿Dónde debería estar el método mostrarInformacion()?
- ¿Qué cosas heredan y cuáles no?

Ironía pedagógica permitida

"Si todos los cazadores hicieran lo mismo, Solo Leveling sería una planilla de Excel con espada."



🔴 PARTE 3 – SISTEMA DE INCURSIONES

Situación

La Asociación de Cazadores decidió organizar mejor las incursiones, porque mandar gente al azar a una mazmorra peligrosa resultó ser una estrategia... discutible.

Ahora vamos a modelar un sistema donde:

- varios cazadores forman un equipo
- una mazmorra tiene enemigos
- el equipo intenta ingresar
- el sistema decide si la incursión es viable

Objetivo técnico

Trabajar:

- relaciones entre múltiples objetos
- composición
- ArrayList
- validaciones
- diseño orientado a responsabilidades

Nuevas clases sugeridas

Enemigo

Atributos sugeridos:

- nombre
- nivel
- vida

Métodos sugeridos:

- mostrarEnemigo()

Equipo

Atributos sugeridos:

- nombre
- ArrayList<Cazador> miembros

Métodos sugeridos:

- agregarCazador(Cazador cazador)
- mostrarMiembros()
- calcularPoderTotal()
- puedeEntrar(Mazmorra mazmorra)

Mazmorra evolucionada

Atributos sugeridos:

- nombre
- dificultad
- ArrayList<Enemigo> enemigos

Métodos sugeridos:

- agregarEnemigo(Enemigo enemigo)
- mostrarEnemigos()
- calcularAmenazaTotal()

Regla del desafío

Un equipo puede entrar a una mazmorra si:

- tiene al menos 3 integrantes
- su poder total es mayor o igual a la dificultad de la mazmorra

Desafío adicional

Podés exigir además que el equipo tenga:

- al menos 1 Tanque
- al menos 1 Sanador

Así evitás que 4 asesinos entusiastas entren a una mazmorra S pensando que la narrativa los va a salvar.

Clase Main sugerida – Parte 3

```
package parte3;

public class MainParte3 {
   public static void main(String[] args) {
      // 1. Crear varios cazadores de distintos tipos
      // 2. Crear un equipo
      // 3. Agregar los cazadores al equipo
      // 4. Crear una mazmorra
      // 5. Crear enemigos y agregarlos a la mazmorra
      // 6. Mostrar el equipo
      // 7. Mostrar la mazmorra y sus enemigos
      // 8. Calcular si el equipo puede entrar
      // 9. Mostrar un mensaje final con el resultado de la incursión
   }

}
```

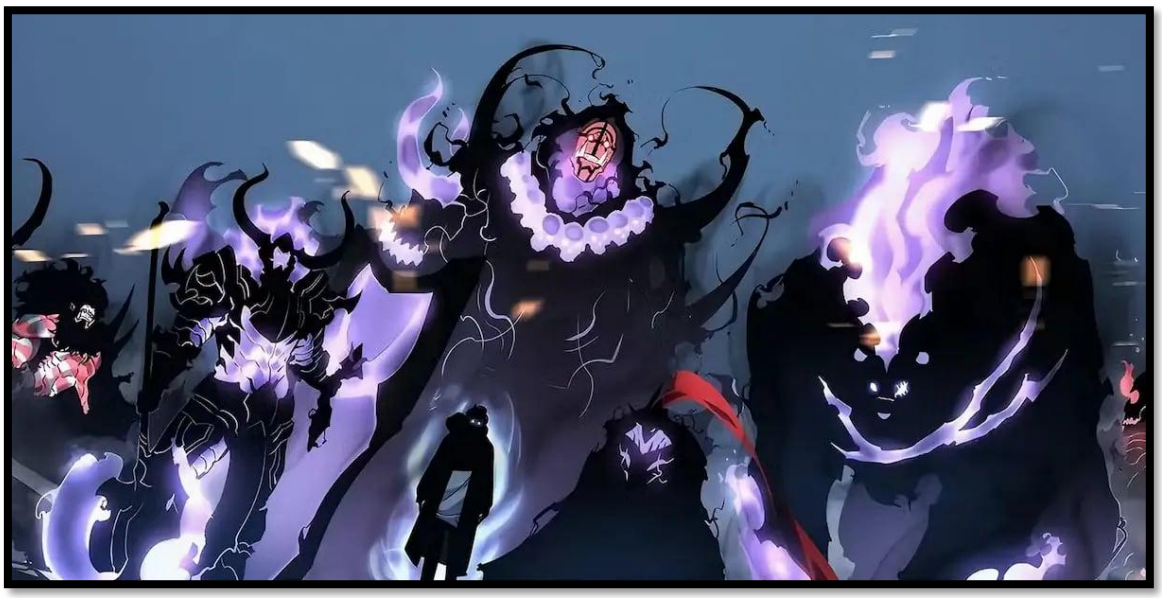
Indicaciones para resolver la Parte 3

1. Crear al menos:
    - 1 equipo
    - 4 cazadores
    - 1 mazmorra
    - 3 enemigos
2. Usar ArrayList en:
    - equipo
    - mazmorra
3. Implementar un criterio para determinar si el equipo puede ingresar.
4. Mostrar una salida clara y narrativa.

Preguntas guía

- ¿Quién conoce a los miembros del equipo?
- ¿Quién calcula el poder total?
- ¿Quién conoce la lista de enemigos?
- ¿La mazmorra debería saber quién entra o eso lo decide el equipo?
- ¿Qué diseño genera menos acoplamiento?

"Entrar a una mazmorra de rango alto con un equipo improvisado no es valentía. Es una forma muy elaborada de desaparecer del sistema."



Criterio pedagógico de progresión

Parte 1

Objetos + relaciones básicas + inventario

Parte 2

Herencia + especialización + cohesión

Parte 3

Colecciones + composición + decisiones de diseño

Este TP permite trabajar explícitamente:

- Programación Orientada a Objetos
- encapsulamiento
- relaciones entre objetos
- herencia
- polimorfismo
- cohesión
- acoplamiento
- uso de ArrayList

"La diferencia entre un programa que funciona y un buen diseño orientado a objetos no está solo en el resultado.
Está en cómo se reparten las responsabilidades."

Soluciones Propuestas:

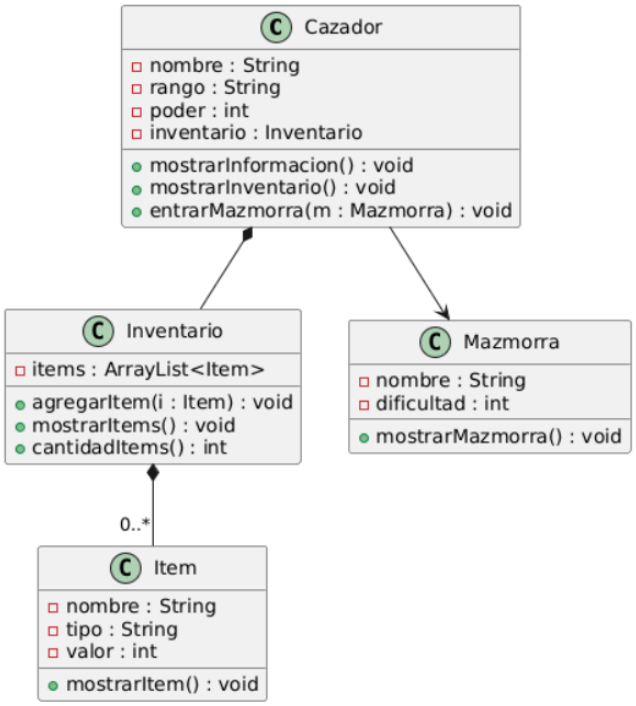
UML – PARTE 1

El despertar del cazador

En esta primera parte el foco está en:

- clases básicas
- atributos
- métodos
- relación entre objetos
- composición simple

El cazador **tiene** un inventario.
El inventario **tiene muchos** ítems.
El cazador además puede entrar a una mazmorra.



UML – PARTE 2

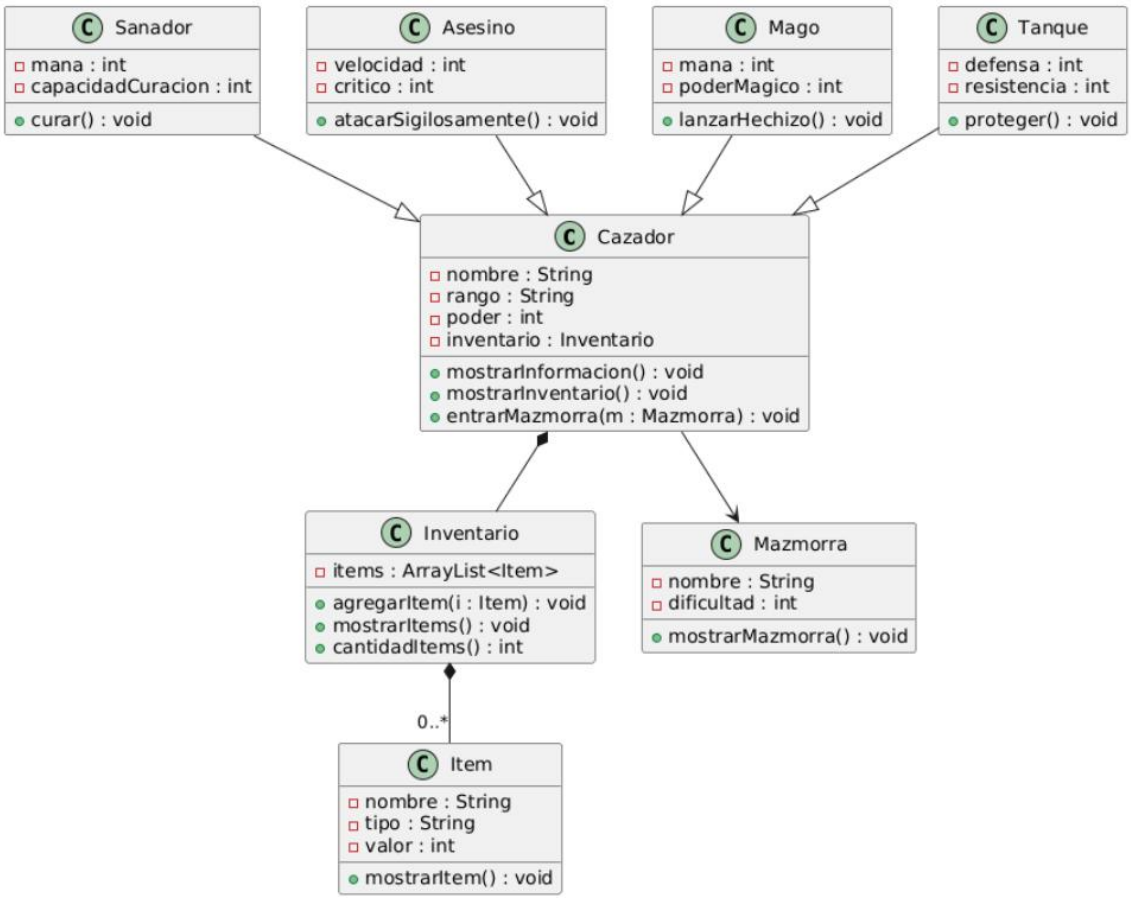
Especialización de cazadores

Acá aparece la evolución fuerte:

- Cazador pasa a ser clase base
- aparecen subclases
- cada tipo de cazador agrega atributos y comportamientos propios

Esto permite introducir:

- herencia
- especialización
- alta cohesión



UML – PARTE 3

Sistema de incursiones

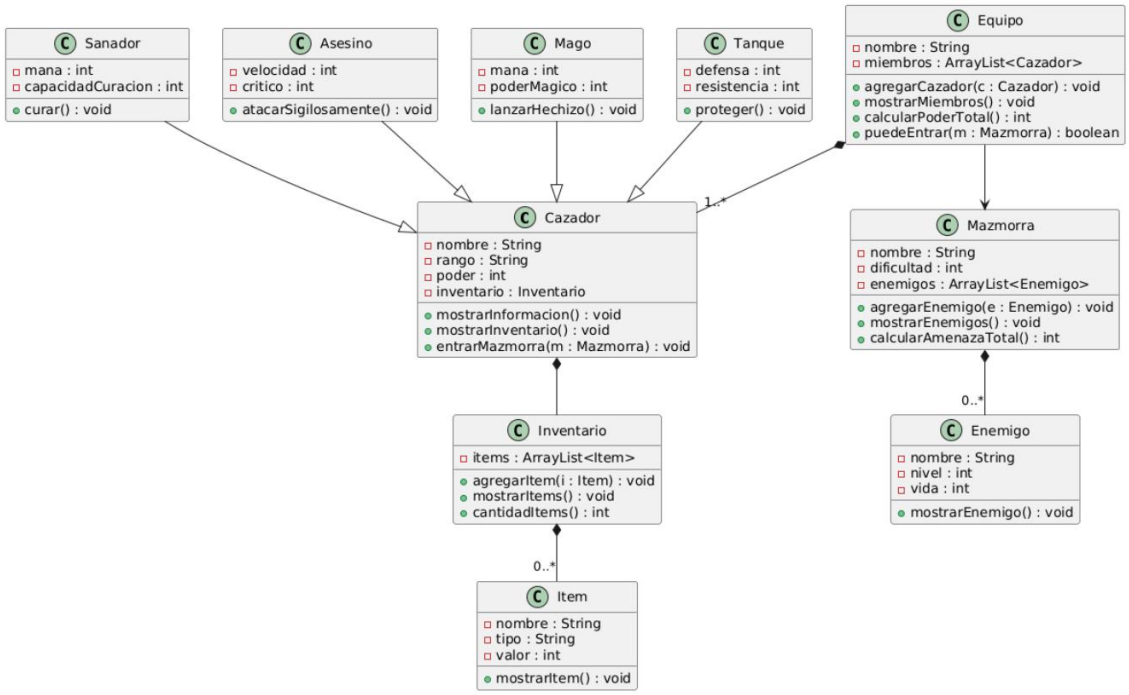
Lectura pedagógica

Acá el sistema se vuelve más interesante:

- un Equipo contiene muchos cazadores
- una Mazmorra contiene muchos enemigos
- el equipo calcula poder total
- la mazmorra calcula amenaza
- aparece más fuerte la idea de **responsabilidades repartidas**

Esta parte es ideal para explicar:

- composición
- colecciones
- cohesión
- acoplamiento



⚔️ **En términos de Solo Leveling**

Al principio eran cazadores rango E:

- código que funciona… más o menos
- todo mezclado
- sobreviviendo por suerte

Ahora ya están empezando a subir de nivel:

- separan responsabilidades
- usan objetos correctamente
- entienden relaciones
- aplican abstracción

👉 Y eso es lo que realmente importa.

Sigan cuestionando sus propios diseños.
Sigan preguntándose *"¿esto debería estar acá?"*.

Porque ahí es donde empieza a aparecer el verdadero aprendizaje.

Y recuerden:

Entrar a una mazmorra sin diseño…
es como entrar a un sistema sin objetos:
puede que funcione… pero no va a durar mucho.